Software Architecture and Design Specification

Project: Web-Based ML Model Evaluator

Version: 1.0

Authors: Vasanth Kumar Reddy, Munireddy, Manoj M S, Sharath H M

Date: 04-10-2026

Status: Final / Academic Submission

Revision	Date	Author	Change Summary
1.0	04-10-2026	Team 16	Architecture and design specification based on the project SRS, STP and supplied architecture template.

Approvals

Role	Name	Signature / Date
Course Coordinator	ASHA K A	
Project Guide	ASHA K A	
Team Lead / Student	VASANTH KUMAR REDDY	

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Web-Based ML Model Evaluator. It translates the project requirements into architecture, component responsibilities, interfaces, security design, runtime flows, error handling and implementation decisions.

1.2 Scope

The system covers authentication, CSV upload and validation, dataset preview/profiling, missing-value and duplicate handling, encoding and scaling, feature/target selection, classification and regression workflows, model selection and hyperparameter configuration, training, testing, evaluation, comparison, visualization, experiment history and result export.

1.3 Audience

Developers, QA engineers, ML/data-science students, researchers, project guides, system administrators and assessment evaluators.

1.4 Definitions

Term	Definition
ML	Machine Learning
SRS	Software Requirements Specification
STP	Software Test Plan
API	Application Programming Interface
UI	User Interface
CSV	Comma-Separated Values
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
ADR	Architecture Decision Record

2. Document Overview

2.1 How to Use This Document

Read the document from architecture goals and constraints through component architecture and design flows. The component diagram describes major boundaries; sequence diagrams describe representative runtime interactions; API design defines logical interfaces; security and traceability connect design decisions to the SRS and STP.

2.2 Related Documents

Document	Purpose
Web-Based ML Model Evaluator SRS v1.0	Functional, non-functional, security, interface and acceptance requirements.
Web-Based ML Model Evaluator STP v1.0	Test scope, strategy, security validation, test cases and traceability.
Requirements Traceability Matrix	Maps requirements to design modules and test cases.

3. Architecture

3.1 Goals & Constraints

- Separate UI, API, data pipeline, ML engine and persistence responsibilities.
- Protect accounts, datasets and experiments using authentication, authorization, secure upload validation, HTTPS/TLS and sanitized errors.
- Do not execute arbitrary user-supplied Python code.
- Handle invalid inputs and training failures without uncontrolled termination.
- Provide visible training progress for long-running operations.

- Routine dataset metadata operations should target a response within 3 seconds under normal test load.
- Initial release primarily supports supervised classification/regression and CSV input.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Normal User	Simple workflow, dataset handling, model configuration and results.
Administrator	User/resource management and monitoring.
Project Guide / Evaluator	Requirement compliance, architecture clarity, testing evidence and acceptance.
Developers	Modularity, maintainability and testability.
QA / Test Engineers	Traceability, security, performance, usability and ML/data correctness.

3.3 Component (UML) Diagram

Figure 1 – Component / Logical Architecture

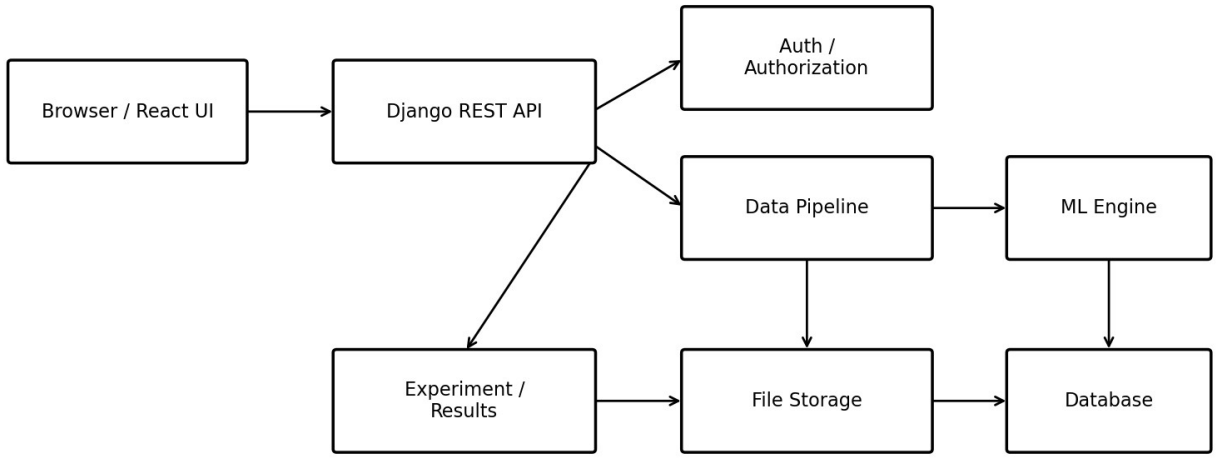


Figure 1. Logical component architecture.

3.4 Component Descriptions

Component	Responsibilities
Browser / React UI	Login, dashboard, upload/preview, preprocessing, model selection, progress, metrics, comparison, visualization and export.
Django REST API	Authentication, authorization, request validation, routing, orchestration and controlled errors.
Authentication & Authorization	Protected sessions, password handling and object-level access control.
Data Pipeline	CSV validation, profiling, cleaning, encoding, scaling and train-test splitting.
ML Engine	Model registry, hyperparameters, training, prediction and classification/regression metrics.
Experiment / Results Service	Experiment metadata, configurations, metrics, comparison and export.
Database	Users, metadata, experiments, configurations and metrics.
File Storage	Uploaded datasets and optional model artifacts.

3.5 Chosen Architecture Pattern and Rationale

A layered web architecture is selected because it provides clear separation of concerns and suits the academic release. React-equivalent UI handles presentation; Django/Django REST Framework provides API/orchestration; data-pipeline and ML-engine modules isolate ML processing; relational database/file storage provide persistence. Microservices are not selected for the initial release because production-scale distributed training is outside scope.

3.6 Technology Stack & Data Stores

Layer	Technology / Option	Purpose
Frontend	JavaScript / React or equivalent	Browser UI.
Backend	Python / Django / Django REST Framework	Web/API/authentication/orchestration.
ML/Data	Scikit-learn, Pandas, NumPy	Algorithms, preprocessing and computation.
Database	SQLite (Active Local/CI) / PostgreSQL or MySQL (Production)	Relational storage for users, datasets, experiment configs, metrics, and results. Zero-config SQLite active for local/CI testing; PostgreSQL/MySQL supported for production (ADR-004).
Visualization	Plotly / Chart.js / Matplotlib	Charts and comparisons.
Source Control	Git / GitHub	Version control and collaboration.

3.7 Risks & Mitigations

Risk	Mitigation
Long ML training	Visible status, controlled datasets and future background workers.
Incorrect metrics	Centralized metric calculation and independent reference checks.
Unsafe uploads	Validate type, size, structure and content before processing.
Cross-user exposure	Object-level authorization.
Dependency mismatch	Record exact versions and reproduce approved environment.

3.8 Traceability to Requirements

SRS Area	Architecture Component(s)
F-001/F-002/F-003	UI + Django API + Authentication/Authorization
F-010 to F-015	UI + API + Data Pipeline + File Storage
F-020 to F-023	UI + API + ML Engine
F-030/F-031/F-034	API + Data Pipeline + ML Engine
F-032/F-033	ML Engine + Results Service
F-040/F-041	Results Service + Visualization + UI
F-042/F-043	Results Service + Database/File Storage
NF-001 to NF-005	API/UI + service boundaries + access control
SR-001 to SR-006	TLS + authentication + validation + authorization + safe processing + sanitized errors

3.9 Security Architecture

Security is based on confidentiality, integrity and safe processing. Production communication uses HTTPS/TLS. Passwords are securely hashed. Uploaded files are validated for type, size and content. Dataset, experiment and model-artifact access is authorized server-side. The workflow does not execute arbitrary Python supplied through datasets, parameters or user input.

STRIDE Threat	Example Risk	Mitigation
Spoofing	Unauthorized login/session use	Authentication and session invalidation.
Tampering	Unauthorized modification	Server-side authorization and controlled persistence.
Repudiation	Actions not attributable to user	Authenticated ownership and metadata.
Information Disclosure	Dataset/internal-error exposure	Object-level authorization and sanitized errors.
Denial of Service	Abusive uploads/processing	Upload limits and validation.
Elevation of Privilege	Access to another user's resources	Authorization on protected resources.

3.10 Deployment View

A browser communicates with the Django API over HTTP(S). The application server hosts the API, data pipeline and ML engine and connects to a relational database and controlled file storage. Production-like deployment uses HTTPS/TLS. Future releases may introduce background workers for long-running jobs.

4. Design

4.1 Design Overview

API endpoints validate input before invoking processing services. The data pipeline creates a reproducible preprocessing and train-test configuration. The ML engine consumes prepared data and model configuration and returns predictions and metrics. Experiment records preserve configuration and results for history, comparison and export.

4.2 UML Sequence Diagrams

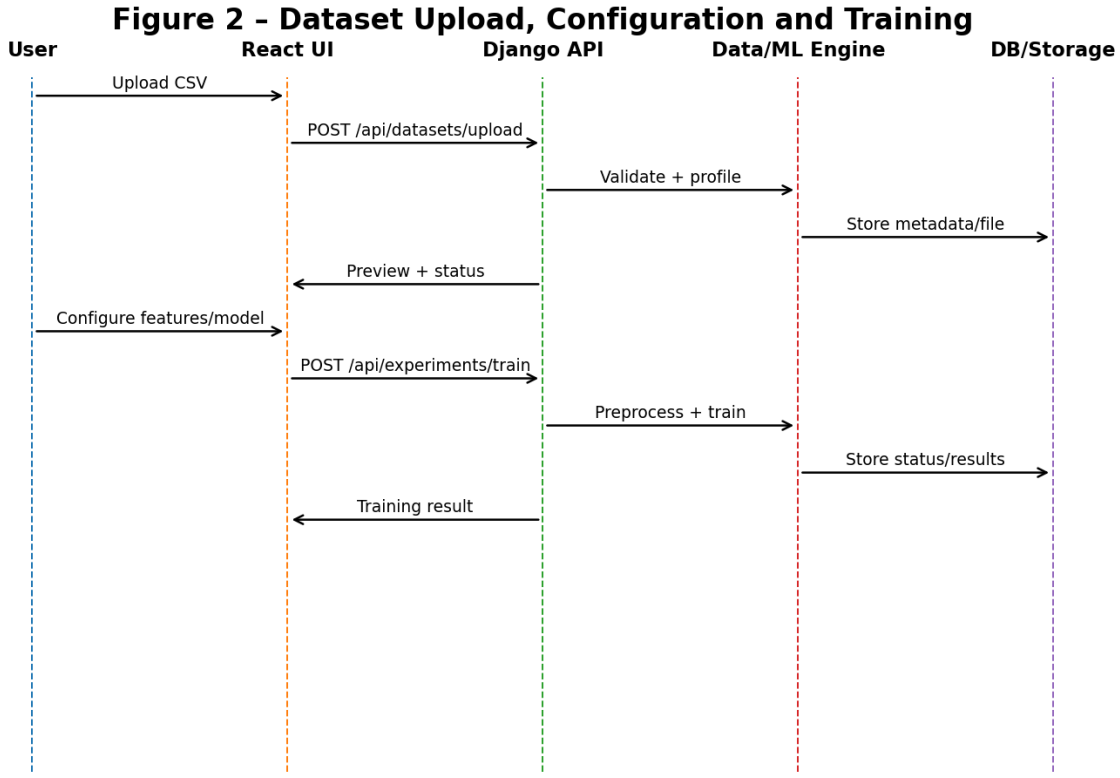


Figure 2. Dataset upload, configuration and training sequence.

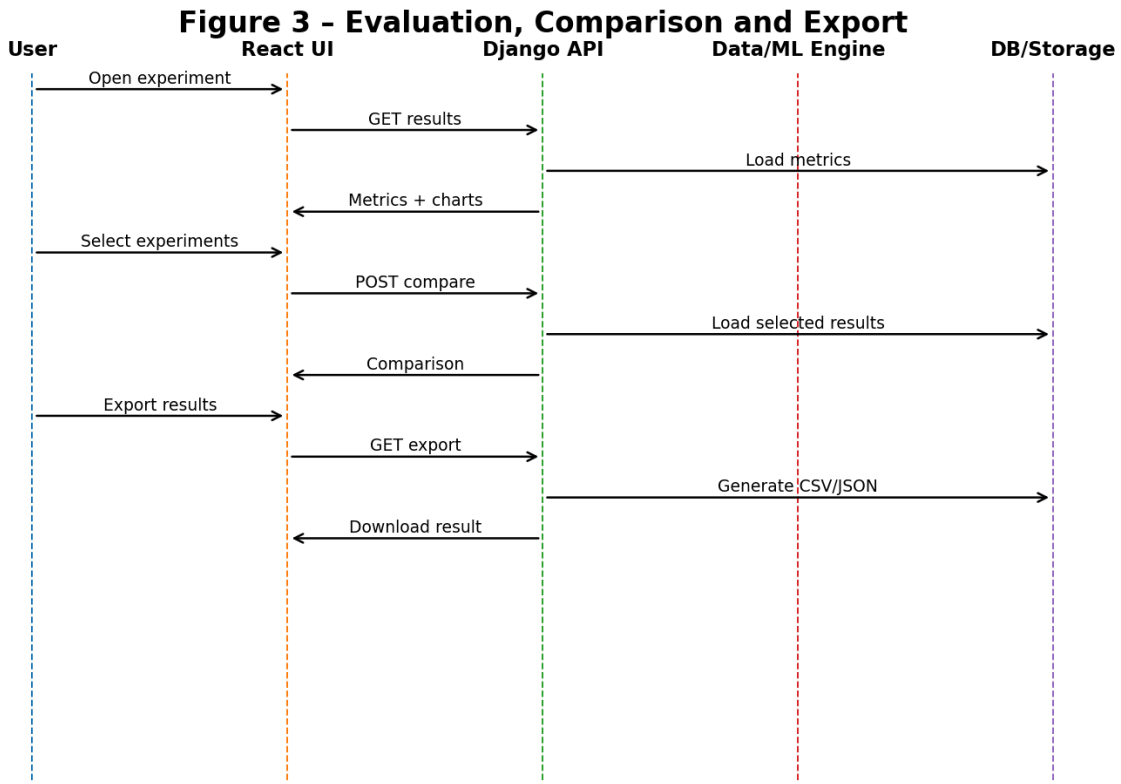


Figure 3. Evaluation, comparison and result export sequence.

4.3 API Design

The API uses authenticated HTTP(S) requests with server-side validation and controlled errors. Endpoint names below are logical interfaces that can be mapped to the final Django URL configuration.

4.3.1 Dataset Management Interface

The Dataset Management component exposes upload and preview operations. The upload interface accepts a supported CSV and authenticated session context, validates type, size and structure, and returns dataset metadata/status. The preview interface returns row/column information, data types and profiling information only for datasets owned by the authenticated user.

4.3.2 Experiment and Evaluation Interface

The Experiment and Evaluation component exposes training, results, comparison and export operations. Requests reference validated dataset/experiment identifiers and supported model configurations. Responses return controlled status, metrics and result data; unauthorized or invalid requests return appropriate controlled errors.

Endpoint	Method	Request	Response / Errors
/api/datasets/upload	POST	Multipart CSV	Dataset ID, validation status, metadata; validation errors.
/api/datasets/{id}/preview	GET	Dataset ID	Rows, counts, types, missing summary; unauthorized/not found.
/api/experiments/train	POST	Dataset, features, target, split, problem type, model, parameters	Experiment ID/status; invalid config/training error.
/api/experiments/{id}/results	GET	Experiment ID	Metrics, predictions and configuration.
/api/experiments/compare	POST	2+ compatible experiment IDs	Comparison data; incompatibility/access errors.
/api/experiments/{id}/export	GET	Experiment ID + CSV/JSON	Machine-readable result file.

4.4 Error Handling, Logging & Monitoring

- Use consistent status codes and application-level error identifiers.
- Do not expose stack traces, secrets, internal paths or sensitive data in user-facing messages.
- Avoid logging plaintext passwords or sensitive dataset contents.
- Record experiment status, configuration and evaluation results.
- Monitor API response time, training failures, upload failures and authorization failures.

4.5 UX Design

- Step-by-step workflow from login through export.
- Dataset preview and profiling before training.
- Visible training status for long operations.
- Readable labels, contrast, keyboard accessibility and responsive layout.
- Appropriate classification and regression metrics with charts/tables.

4.7 Database Architecture & Entity-Relationship Design

The Web-Based ML Model Evaluator employs a normalized relational database design structured around three core entities: User accounts (auth_user), uploaded datasets (datasets_dataset), and executed ML experiments (experiments_experiment). The architecture enforces strict referential integrity, foreign key constraints, and cascade deletion rules to maintain data consistency.

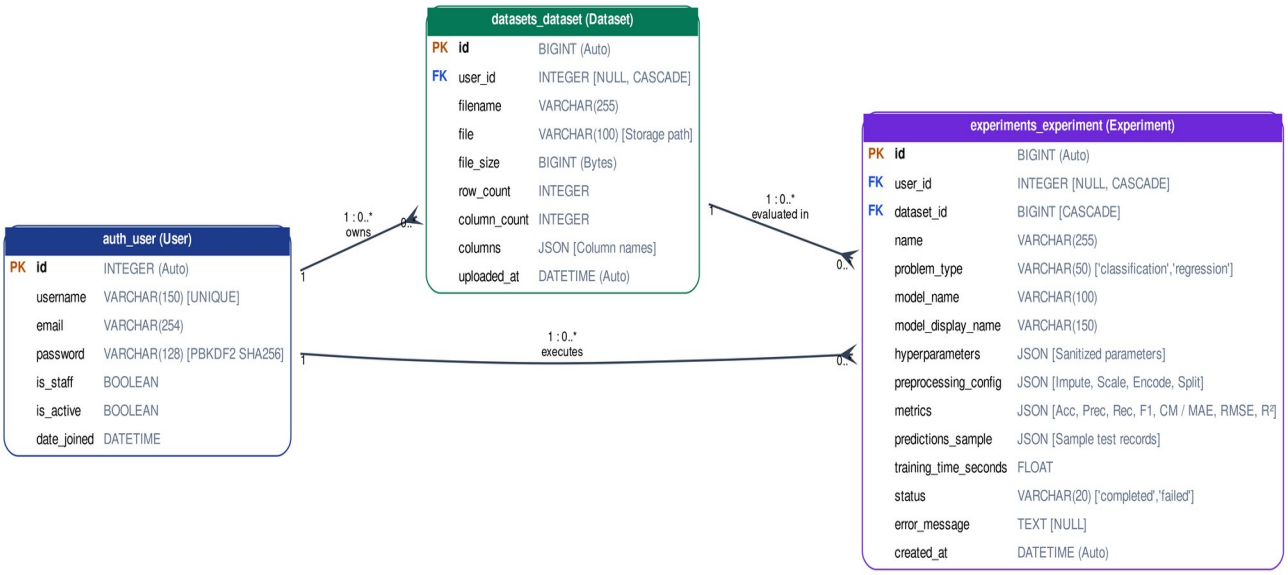


Figure 4. Entity-Relationship Diagram for Web-Based ML Model Evaluator.

4.7.1 Data Schema Specifications

1. auth_user (User Entity): Managed via Django authentication framework. Stores credentials using PBKDF2 with SHA-256 password hashing. Core fields include id (PK, Integer), username (VARCHAR(150), Unique), password (VARCHAR(128)), email (VARCHAR(254)), is_staff (Boolean), and date_joined (DateTime).
2. datasets_dataset (Dataset Entity): Represents validated tabular CSV datasets. Core fields include id (PK, BigAutoField), user_id (FK referencing auth_user.id, Nullable, ON DELETE CASCADE), filename (VARCHAR(255)), file (VARCHAR(100), relative path in media storage), file_size (BigInteger bytes), row_count (Integer), column_count (Integer), columns (JSONField column name list), and uploaded_at (DateTime).
3. experiments_experiment (Experiment Entity): Preserves complete execution records, configurations, evaluation metrics, and prediction samples. Core fields include id (PK, BigAutoField), user_id (FK referencing auth_user.id, Nullable, ON DELETE CASCADE), dataset_id (FK referencing datasets_dataset.id, ON DELETE CASCADE), name (VARCHAR(255)), problem_type (VARCHAR(50), choices: 'classification', 'regression'), model_name (VARCHAR(100)), model_display_name (VARCHAR(150)), hyperparameters (JSONField), preprocessing_config (JSONField), metrics (JSONField

storing Accuracy, Precision, Recall, F1, Confusion Matrix, ROC-AUC or MAE, MSE, RMSE, R^2), predictions_sample (JSONField), training_time_seconds (Float), status (VARCHAR(20)), error_message (TextField), and created_at (DateTime).

4.7.2 Cardinality & Referential Integrity

• User to Dataset (1 : 0..*): A user may upload zero or many datasets. Each dataset optionally belongs to one user. When a user account is deleted, all associated datasets and stored files are removed via cascading delete.
• User to Experiment (1 : 0..*): A user may execute zero or many experiments. Each experiment belongs to one user.
• Dataset to Experiment (1 : 0..*): A dataset may serve as the training baseline for zero or many experiments. Deleting a dataset cascades to delete all experiments trained on that dataset.

4.6 Open Issues & Next Steps

- Finalize exact frontend and package versions used in implementation.
- [Resolved] Finalized and documented relational schema in Section 4.7. Verified dual-engine configuration: SQLite operates as the active, zero-configuration engine for local evaluation and GitHub Actions CI; PostgreSQL/MySQL verified for production multi-worker deployment (ADR-004, Deviation 1).
- Introduce background workers if long-running training becomes a limitation.
- Future enhancements may include GPU/deep learning, AutoML, tuning, clustering, XAI and model deployment subject to revised requirements.
- Update SRS/STP RTM status during final implementation and testing; this design document does not claim that tests have passed.

5. Appendices

5.1 Glossary

Term	Meaning
API	Application Programming Interface
ADR	Architecture Decision Record
CSV	Comma-Separated Values
ML Engine	Model configuration, training, prediction and evaluation module
RTM	Requirements Traceability Matrix
SRS	Software Requirements Specification
STP	Software Test Plan
UAT	User Acceptance Testing

5.2 References

- Web-Based ML Model Evaluator – SRS v1.0, dated 04-09-2026.
- Web-Based ML Model Evaluator – STP v1.0, dated 17-09-2026.
- IEEE 42010 – Architecture descriptions for systems and software.
- OWASP application security guidance.
- NIST SP 800-160 – Systems Security Engineering guidance.
- Django, Django REST Framework, Scikit-learn, Pandas, NumPy and database documentation.

5.3 Tools

- PlantUML / draw.io – UML diagrams and architecture visualization.
- Git / GitHub – source control and collaboration.
- Postman / Insomnia – API verification.
- pytest / Django TestCase – automated unit/integration testing.
- Selenium / Playwright – browser workflow testing where applicable.

5.4 Architecture Decision Records (ADRs)

ADR	Decision	Rationale	Alternative
ADR-001	Layered architecture	Clear separation and	Microservices rejected for initial

		maintainability.	scale.
ADR-002	Django REST backend	Matches SRS stack and supports API/auth.	Other frameworks not required.
ADR-003	Classical supervised ML	Matches initial scope.	Deep learning/AutoML outside initial scope.
ADR-004	Relational persistence with dual-engine support (SQLite active for local/CI; PostgreSQL/MySQL for production)	Enforces relational integrity across users, datasets, and experiments. SQLite ensures zero-configuration onboarding and fast automated CI; standard Django ORM ensures full production portability.	Distributed NoSQL not required.
ADR-005	No arbitrary user code	Reduces security risk.	Executing user Python prohibited.

5.5 Design-to-Test Traceability Summary

Design Area	Representative STP Tests
Authentication & Authorization	TC-AUTH-01 to TC-AUTH-03; TC-SEC-01 to TC-SEC-03, TC-SEC-05
Dataset/Data Pipeline	TC-DATA-01 to TC-DATA-06; TC-SEC-04
Model Selection/Configuration	TC-ML-01 to TC-ML-04
Training/Testing	TC-TRAIN-01 to TC-TRAIN-03; TC-UX-01
Evaluation	TC-EVAL-01 and TC-EVAL-02
Comparison/Visualization	TC-COMP-01 and TC-COMP-02
History/Export	TC-HIST-01 and TC-EXP-01
Performance/Reliability/Accessibility	TC-PERF-01; TC-REL-01; TC-UX-02
Security Architecture	TC-SEC-01 to TC-SEC-07